Matteo Muraca, mat. 201074

Forest a green cellular automata

Lo scopo di questo progetto è di implementare un automa cellulare in C++, utilizzando il calcolo parallelo per migliorare le prestazioni.

Un automa cellulare è un modello matematico usato per descrivere l'evoluzione di un sistema complesso. Ogni CA è formato da una griglia n-dimensionale, costituita da celle, di dimensione finita, ed ogni cella può assumere un numero finito di stadi.

Essi vengono impiegati per rappresentare l'evoluzione globale di fenomeni che dipendono solo da leggi locali, come il comportamento dei gas perfetti, l'evoluzione di una popolazione, un ecosistema, trovando spazio in campi quali biologia, fisica e statistica.

Forest è un CA bidimensionale che simula la propagazione degli incendi in una foresta: viene naturale supporre che l'unità fondamentale sia l'albero.

Ogni albero assume uno stadio tra:

- *Vivo*, rappresentato dal colore verde;
- *In fiamme*, rappresentato dal colore rosso;
- *Morto*, rappresentato dal colore marrone.

Un albero vivo può essere colpito da un fulmine, e dunque prendere fuoco, con probabilità 1/l. Un albero può crescere in uno spazio vuoto con probabilità 1/g.
I valori di default per queste due costanti sono l = 2000, g = 200, ma possono essere modificati a piacimento semplicemente modificando le apposite variabili nel codice, come anche la dimensione della griglia, rappresentata da una matrice.

Oltre a queste regole legate alla probabilità, un albero prende fuoco se almeno uno dei suoi quattro vicini (sopra, sotto, destra e sinistra) è in fiamme.
Ogni albero in fiamme al turno successivo sarà sostituito da una cella marrone.

Il progetto comprende:

- *serialForest.cpp* - versione seriale dell'automa;
- *parallelForest.cpp* - versione parallela dell'automa in MPI;
- *groupForest.cpp* - doppio automa che sfrutta i gruppi e i comunicatori di MPI.

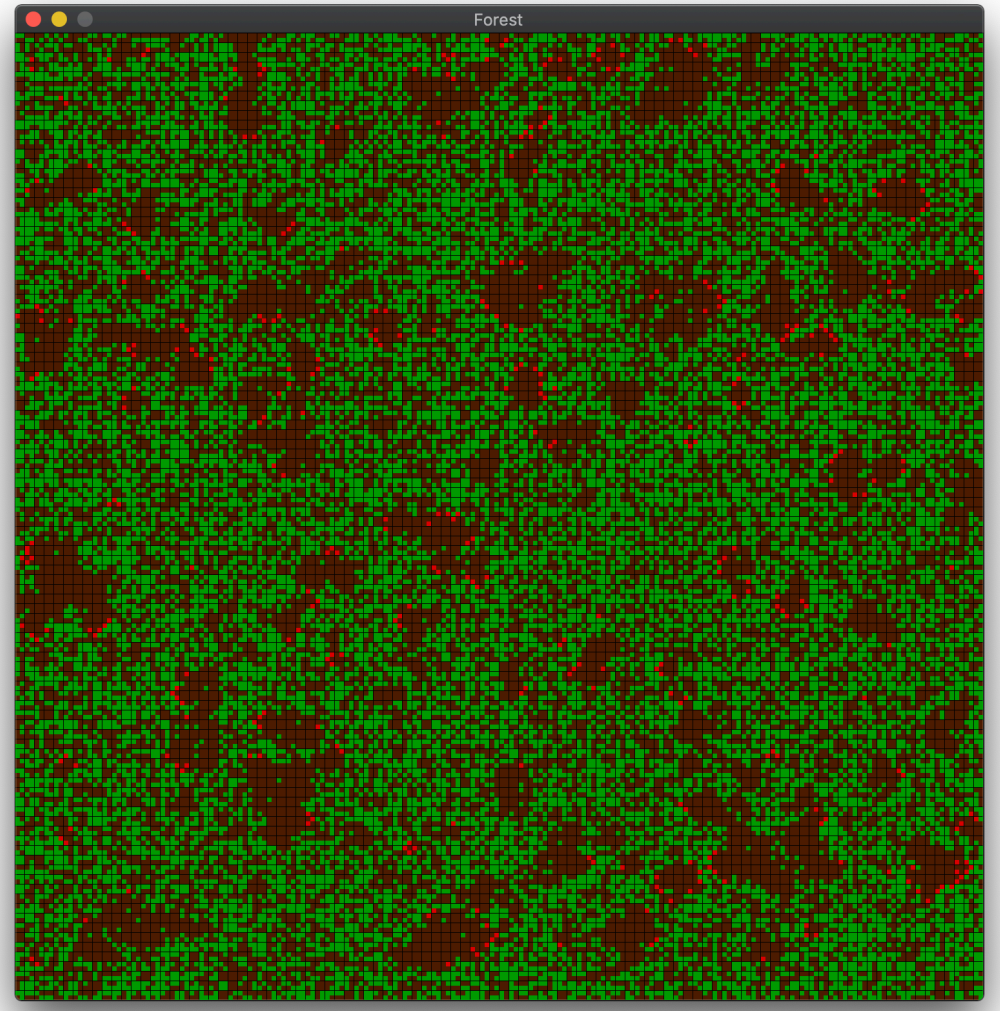
Inoltre, nella cartella test:

- *testForest.cpp* - versione utilizzata per eseguire i test con gli script Python;
- *test.py, test.sh, hostfile* - script Python per i test.

serialForest.cpp

In questo file c'è una prima versione seriale dell'automa cellulare.
Lo scopo della parallelizzazione è quello di migliorare le prestazioni rispetto al codice seriale, dunque viene naturale scrivere prima di tutto un codice di questo tipo, per capire anche come funzionano gli algoritmi utilizzati e in che modo si può parallelizzare, scegliendo paradigmi e strumenti adatti.

Innanzi tutto vengono allocate due matrici dinamiche di dimensioni uguali: per ogni step, una delle due rappresenterà lo stato corrente, mentre nell'altra verrà calcolato lo stadio successivo. Infatti, la prima sarà inizialmente riempita in maniera casuale con 0 ed 1, rappresentanti rispettivamente una cella morta ed un albero.
Viene dunque richiamato il codice di inizializzazione di Allegro5, libreria utilizzata come motore grafico, per creare una finestra, attivare la tastiera e disegnare la prima matrice.
In seguito all'inizializzazione della variabile booleana whatMatrix, che serve per sapere quale delle due matrici contiene lo stato corrente e in quale bisogna calcolare il nuovo, si entra in un ciclo while, dal quale si uscirà premendo il tasto esc. In questo while, un ciclo for scorre tutta la matrice corrente, per calcolare sulla nuova matrice il nuovo stadio, che verrà disegnato sulla finestra.



parallelForest.cpp

A partire dal programma seriale, ho scritto un programma parallelo utilizzando MPI (Message Passing Interface) ed in particolare l'implementazione OpenMPI per C++. MPI è un protocollo di comunicazione per i sistemi a memoria distribuita, ed è lo standard nell'industria moderna in quanto ha un'alta portabilità ed ottimizzazione.
Come dice il nome stesso, MPI è basato su invio e ricezione di messaggi, sia in maniera point-to-point che in maniera collettiva, ed entrambe sono state utilizzate nella realizzazione di questo progetto.

Come nel programma seriale, il processo principale (root) ha una matrice dinamica riempita in maniera casuale da 0 ed 1, richiama le funzioni di avvio di Allegro5, crea la finestra e disegna la matrice.
La differenza sostanziale tra la versione seriale e la parallela sta nel modo in cui la nuova matrice viene calcolata: non ci sono più due matrici, ma due sottomatrici per ogni processo, per le quali la dimensione delle righe viene calcolata come numero di righe della griglia diviso numero di processi. In questo modo, però, è necessario che la dimensione della griglia sia divisibile per il numero di processi: questo controllo avviene in riga 190, prima delle varie inizializzazioni. Si può notare che è solo il root a fare questi controlli, come è solo il root a disegnare la matrice: nel primo caso è perché ci potrebbero essere errori nella scrittura dell'output, che verrebbe scritto più volte, nel secondo perché altrimenti si vedrebbero più finestre nello stesso momento.

La comunicazione collettiva è fondamentale qui, perché viene utilizzata nelle operazioni di scatter e gather. Lo scatter è il processo di suddivisione di un'area contigua di memoria in diverse parti di uguali dimensioni, una per ogni processo. Un'operazione di scatter è svolta all'inizio del codice, dove la matrice principale viene suddivisa tra i vari processi, in maniera molto più veloce rispetto al singolo invio delle porzioni della matrice dal processo principale a tutti gli altri. Al termine di questa operazione, tutti i processi avranno nella prima sottomatrice una porzione della griglia, per poter calcolare nella seconda il nuovo stadio dell'automa.
Al contrario dello scatter, il gather è l'operazione di ricomposizione di un insieme di dati contenuto nelle memorie dei vari processi: esso viene richiamato al termine di ogni step del ciclo while per ricomporre all'interno del processo root la matrice, così da poterla disegnare. Non è necessario fare un nuovo scatter all'inizio del ciclo, poiché ogni processo ha già in una delle due sottomatrici lo stadio corrente, dunque si utilizza, allo stesso modo del programma seriale, la variabile booleana whatMatrix per sapere in quale delle due è contenuto.

In questo modo, la nuova matrice viene calcolata, ma non in maniera corretta, in quanto una cella dipende da tutti e quattro i suoi vicini, dunque le righe di confine superiore non sanno cosa accade sopra di esse, e le righe di confine inferiore non sanno cosa accade sotto di esse. È qui che entra in gioco la comunicazione punto a punto: ogni processo invia al processo di rank precedente al proprio il primo vettore della propria matrice, e riceverà dal processo successivo il lowerVector,

poi invierà al processo di rank successivo l'ultima riga della propria matrice, e riceverà quello che sarà l'upperVector dal processo di rank precedente.
Ovviamente, i processi con rank minimo e massimo non avranno un processo con rank rispettivamente precedente e successivo, dunque riempiranno questi vettori con 0, che rappresenta una cella vuota.
In questo modo, i processi sapranno ciò che succede al confine, e il nuovo stadio dell'automa potrà essere calcolato in maniera corretta.

Per poter uscire dal programma premendo esc, il while richiama la funzione continueProcessing, nella quale il processo root controlla lo stato corrente della tastiera: se il tasto esc è stato premuto, invierà a tutti i processi un messaggio contenente il valore INT_MAX, altrimenti invierà un messaggio contenente 0. Tutti i processi continueranno nel while se il messaggio è diverso da INT_MAX, altrimenti procederanno all'uscita, per la deallocazione della memoria e la finalizzazione.

groupForest.cpp

MPI possiede delle funzionali avanzate per ampliare le possibilità di comunicazione: i messaggi, sia diretti che collettivi, vengono inviati tra processi che fanno parte dello stesso comunicatore, che forma una sorta di rete teorica tra i processi di un gruppo. Il comunicatore di default si chiama MPI_COMM_WORLD, e contiene tutti i processi che il sistema operativo assegna al programma in esecuzione.
Si possono creare dei nuovi comunicatori, a partire da sottogruppi di world, ed è proprio su questo che si basa l'idea di questo codice: creare due gruppi diversi, con due comunicatori associati, per poter calcolare e visualizzare due automi in contemporanea.

L'utente ha la possibilità di scegliere quanti e quali processi faranno parte del primo gruppo, tramite l'input da terminale. Sarà il root a inviare i dati a tutti gli altri processi per poter formare e utilizzare questi gruppi. Il codice funziona esattamente come il parallelForest, con la differenza che ci saranno due processi root, uno per ogni automa, che disegneranno i due diversi automi in due finestre diverse.
Turno per turno, l'utente potrà vedere in output il tempo di calcolo per le singole matrici, per comprendere in runtime le potenzialità del calcolo parallelo. Lo scopo di questo programma, infatti, è di mostrare in tempo reale quali sono le differenze di tempi e di velocità utilizzando un numero diverso di processi per ogni automa.

Nelle successive pagine sono riportati i test effettuati su varie macchine.

MacBook Pro 2019

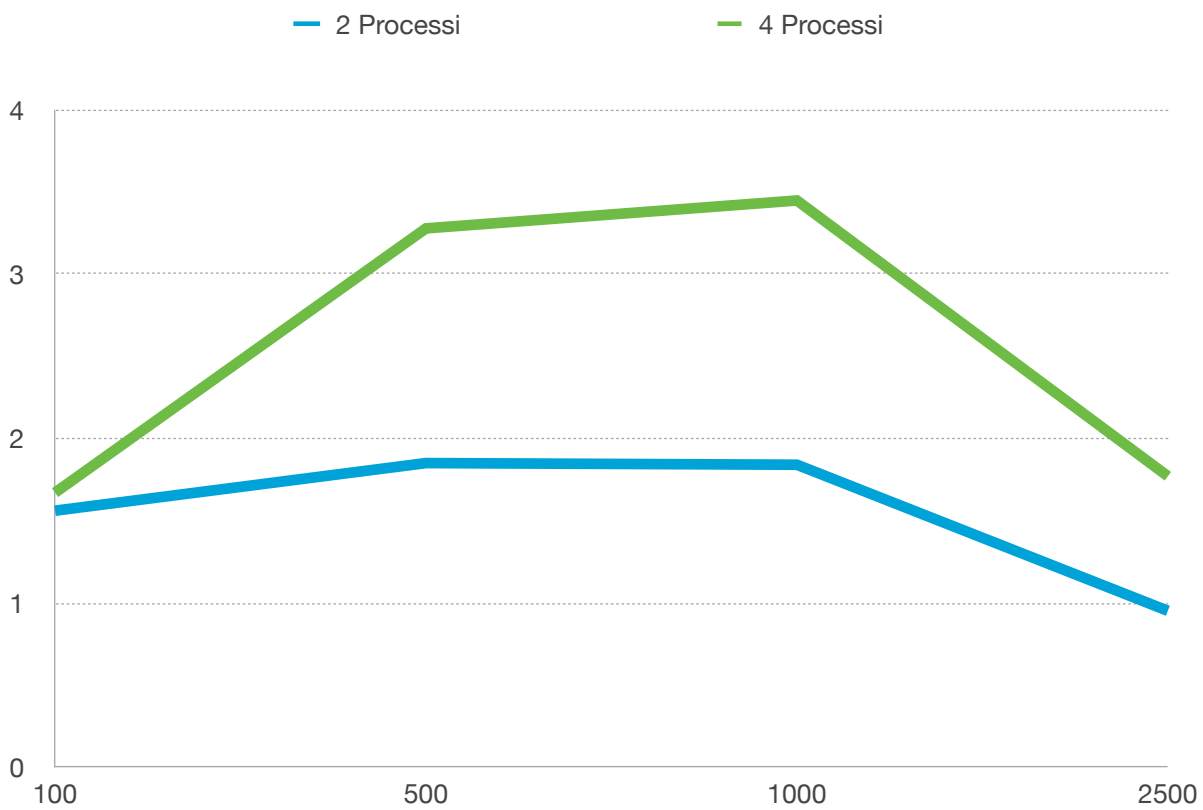
Intel Core i5-8257U, 4 cores & 8 threads, up to 3.9GHz, RAM 8GB
MacOS 10.15.5

Tempo di processamento medio in secondi - 1000 steps - MBP

dim	Seriale	MPI - 2 Processi	MPI - 4 Processi
100	0.25	0.16	0.15
500	3.51	1.90	1.07
1000	14.03	7.65	4.07
2500	88.14	93.18	49.78

Speedup medio - MBP

dim	Seriale	MPI - 2 Processi	MPI - 4 Processi
100	1	1.56	1.67
500	1	1.85	3.28
1000	1	1.84	3.45
2500	1	0.95	1.77

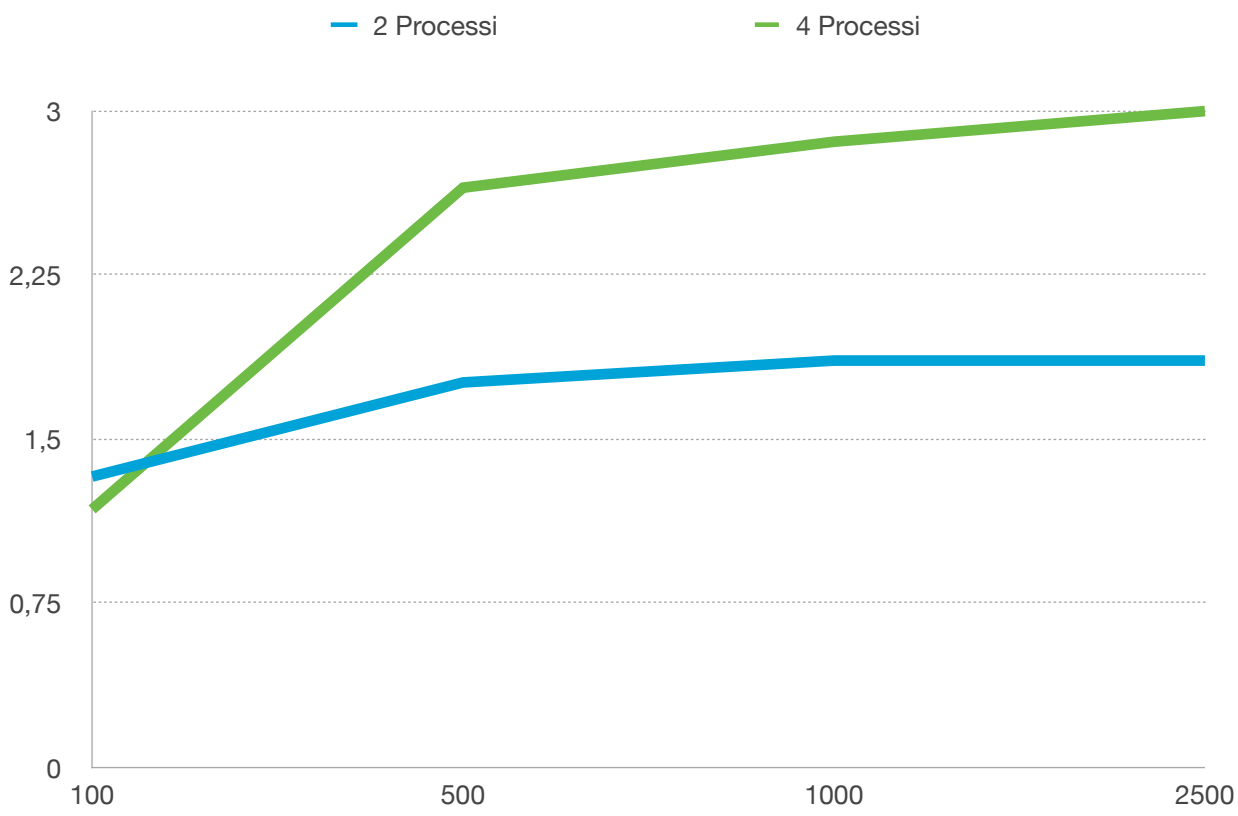


MSI-GS65 8RF

Intel Core i7-8750H, 6 cores & 12 threads, up to 4.1GHz, RAM 16GB
ManjaroLinux on 5.6.18 kernel

Tempo di processamento medio in secondi - 1000 steps - MSI

dim	Seriale	MPI - 2 Processi	MPI - 4 Processi
100	0.20	0.15	0.17
500	1.72	0.98	0.65
1000	6.71	3.60	2.35
2500	43.01	23.19	14.31

Speedup medio - MSI

dim	Seriale	MPI - 2 Processi	MPI - 4 Processi
100	1	1.33	1.18
500	1	1.76	2.65
1000	1	1.86	2.86
2500	1	1.86	3.00